

RALPH LOOP

从一个 while 循环到生产级长运行 Coding Agent

核心原理 • Fresh Context • External Memory • Backpressure • Verification • Stop Condition
• Graph of Loops

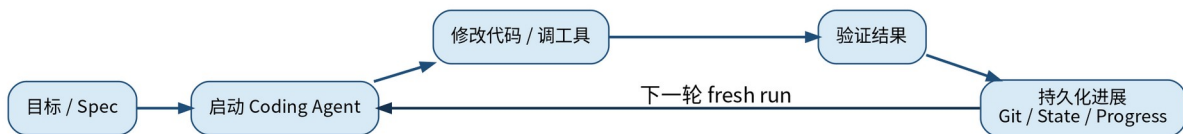


图 1 Ralph 的最小闭环：目标 → Agent → 工作 → 验证 → 持久化 → 新一轮

一句话结论

Ralph 的关键并不是“把 Agent 放进 while true”，而是把人工逐轮催促变成一个外部可编程闭环：每轮 fresh context，只完成一个小任务，把真正进展写入外部状态，再用可执行证据决定继续、提交、回滚或停止。

技术研究报告 • 2026 年 8 月

执行摘要

Ralph Loop 是 Geoffrey Huntley 推广的一种 autonomous coding pattern。最初的外观极其简单：反复运行 Coding Agent；但真正让它从“死循环烧 Token”升级为工程方法的，是一组相互配合的约束：一个任务一轮、稳定规格说明、fresh context、外部持久状态、自动验证、明确停止条件，以及根据失败行为持续调优 harness。原始 Ralph 强调单仓库、单进程、one task per loop，并把规格和计划稳定地重新注入每轮上下文。
[1]

随后公开实现（如 snarktank/ralph）把这套思想进一步工程化：每个 iteration 启动新的 AI 实例；跨轮记忆通过 Git、progress.txt、prd.json 以及 AGENTS.md 等外部载体维持；每轮选择一个未完成 story，执行质量检查，通过后 commit 并更新状态，直到所有 story 通过或达到最大迭代数。
[2]

2026 年的 Loop Engineering 研究进一步把这类系统抽象为“loop specification”：trigger、goal、verification、stopping rule、memory 等组成一个独立于模型内部 tool-use cycle 的外部控制层。
[3] 这为 Ralph 提供了更严格的工程语言：它不是普通 retry，也不是 Agent 内部的 think-act-observe，而是包裹在 Agent harness 外部、对生命周期负责的循环。

本报告的判断

Ralph 的能力上限通常不由 Generator 决定，而由 Verifier 和 State Design 决定。模型再强，如果验收标准容易假阳性，循环只会更快地把错误固化；反过来，即使单轮模型不完美，只要任务足够小、验证足够强、检查点足够清晰，系统仍可通过多轮获得稳定进展。

核心六要素

要素	作用	缺失后的典型退化
Goal / Spec	给循环稳定目标与约束	循环持续“很努力”，但方向漂移
Small Task	把每轮搜索空间限制在可完成范围	半成品堆积、上下文爆炸
Fresh Context	消除历史噪声与 context rot	旧日志/失败方案持续占用注意力
Persistent State	把真正进展跨轮保存	每轮重新发现、重复劳动
Backpressure / Verification	把错误转成机器可读反馈	错误被当成成功并继续复利
Stop Condition	定义 DONE、RETRY、HUMAN、BUDGET 等终止态	while true 无界消耗

目录

- 1. Ralph 的起源与准确定位**
2. 三种“循环”必须先分清
3. 最小 Ralph：为什么 while loop 只是外壳
4. One Item Per Loop：控制单轮搜索空间
5. Fresh Context：把 iteration boundary 当作 Context GC
6. External Memory：让环境记住，而不是让对话记住
7. Backpressure 与 Verifier：真正的质量控制面
8. 一轮 Ralph 的完整执行逻辑
9. Stop Condition、预算与生命周期状态
10. 为什么 Ralph 会有效：从控制系统角度理解
11. Ralph 的主要失败模式
12. 从 Pure Ralph 到 Production Loop
13. Ralph 与 Graph Engineering：Graph of Loops
14. 实战设计清单与推荐架构
- 15. 结论与参考资料**

1. Ralph 的起源与准确定位

Ralph Wiggum pattern 由 Geoffrey Huntley 在 2025 年系统传播。原始文章对它的定位并不是复杂的 multi-agent 编排，而是“monolithic”：在单一 repository 中，以单一主进程反复工作，并强调每个 loop 只处理一个任务。[1] 这与当时大量把不确定 Agent 拆成多个互相通信服务的思路形成鲜明对照。

因此，Ralph 的“简单”是有意为之：先把最关键的反馈闭环做稳，再谈更复杂的多 Agent 拓扑。Huntley 后续又把 Ralph 描述为一种 orchestrator pattern：稳定分配所需规格与目标，观察循环中的失败域，然后把修复经验编码回循环，让同类失败尽量不再发生。[5]

1.1 不应把 Ralph 等同于某个具体仓库

需要区分“Ralph pattern”和“某个 Ralph implementation”。前者是一套方法论；后者是脚本、PRD 格式、CLI、Agent tool、插件和目录结构的具体实现。例如 snarktank/ralph 使用 prd.json、progress.txt、Git history 和 fresh AI instance，这是一种非常清晰的工程化实现，但并非 Ralph 唯一合法形态。[2]

1.2 Ralph 的核心对象是外部控制循环

```
while not done:
    context = build_context(spec, repo, progress, tests)
    result = run_coding_agent(context)
    evidence = verify(result)
    state = update_state(result, evidence)
    done = should_stop(state, evidence, budget)
```

上面的伪代码比“while true; agent”更接近 Ralph 的真实本质：循环体中最重要的是 context reconstruction、verification、state update 和 stop decision，而不仅是模型调用。

2. 三种“循环”必须先分清

讨论 Ralph 时最常见的概念混淆，是把三种不同层级的循环都叫“Agent Loop”。如果不先拆开，就很难理解 Ralph 到底新增了什么。

层级	典型形式	状态是否演进	Ralph 是否等同
普通程序 Retry	失败后重试同一个 API / command	通常很少	否
Agent 内部 Tool-Use Loop	LLM → Tool → Observation → LLM	会在同一 session 内累积	否，这是 harness 的内部 plumbing
Ralph 外层 Loop	启动 Agent → 工作 → 验证 → 保存 → 新 Agent	跨 iteration 持久演进	是

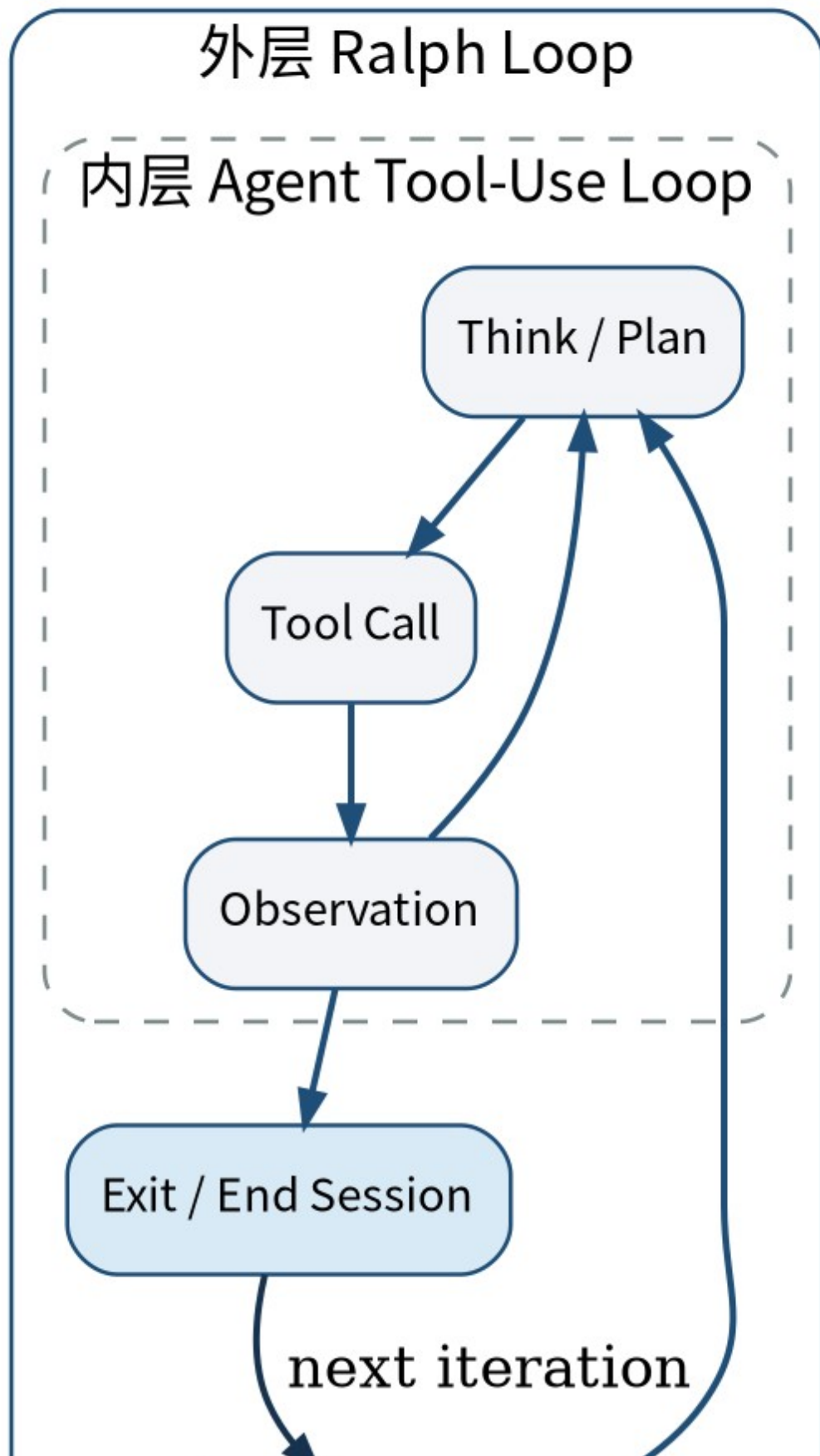


图 2 Ralph 外层 loop 包裹 Coding Agent 内部 tool-use loop

2026 年的 Loop Engineering 论文也明确区分外部 loop specification 与 harness 内部的 perceive-act-observe cycle：前者负责“为什么启动、追什么目标、怎样验收、何时停止、记住什么”，后者只是 Agent 能够调用工具和观察结果的内部执行机制。[3]

3. 最小 Ralph：为什么 while loop 只是外壳

最小 Ralph 常被演示为 shell loop，因为它用极少代码揭示了一个重要事实：一次 Agent session 不需要承担整个项目生命周期。Session 可以结束，工作状态仍留在 repository；外层系统再启动新的 session 继续即可。

```
# 教学级最小模型（概念示意）
while true; do
  coding-agent < PROMPT.md

done
```

但如果只有这一行，系统实际上缺少四件生产必需品：成功证据、持久任务状态、失败分类、资源边界。因此 Pure Ralph 适合说明思想，不足以直接成为企业级 runtime。

3.1 Stateful Iterative Improvement，而不是 Stateless Retry

普通 retry 近似于反复调用 $f(x)$ ；Ralph 则让环境状态 S 在每轮后改变。正确抽象是 $S_t \rightarrow \text{Candidate}_{t+1} \rightarrow \text{Evidence}_{t+1} \rightarrow S_{t+1}$ 。下一轮 Agent 面对的是已经被修改、提交、测试过的新世界，而不是相同输入。

关键区别

Retry 关心“同一个操作再试一次”；Ralph 关心“基于上一轮产生的新代码、新证据和新经验，再进行下一步状态转移”。

4. One Item Per Loop：控制单轮搜索空间

Huntley 在原始 Ralph 文章中反复强调 one item per loop，并指出上下文窗口是稀缺资源：单轮工作越大，越容易让搜索、实现、测试和日志同时竞争注意力。[1] 公开实现也把 story 大小视为关键设计约束：每个 PRD item 应足够小，使一个 context window 内能够完成。[2]

坏的任务粒度	更合适的拆分
“增加完整鉴权系统”	DB migration → User model → Login API → Login UI → Session test
“重构整个 API”	先限定一个 endpoint / module / dependency boundary
“完成整个 Dashboard”	按数据源、组件、交互、E2E 分成独立 story
“修所有测试”	按失败簇或根因拆分，避免一轮覆盖多个 failure domain

从搜索问题角度看，one item per loop 的作用类似于缩小 action space：Agent 不再同时决定几十个目标的优先级、实现顺序和验收方式，而只需在一个局部问题中完成闭环。

5. Fresh Context: 把 iteration boundary 当作 Context GC

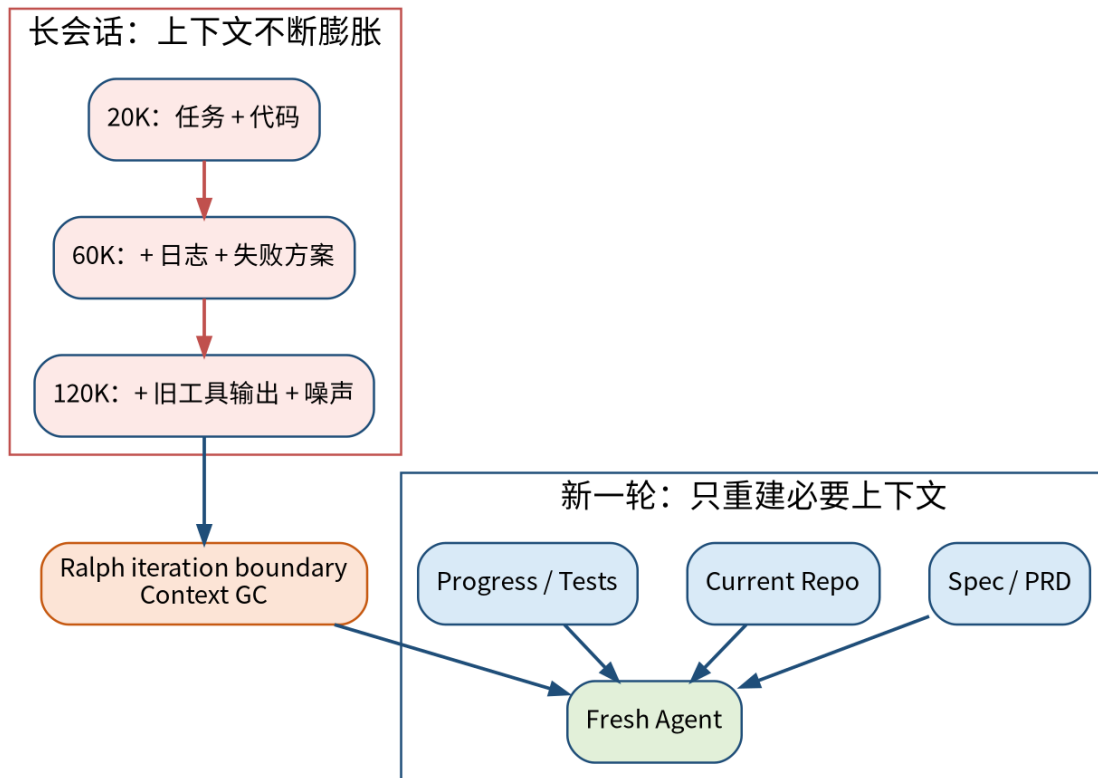


图 3 Fresh Context 的直观含义：在 iteration 边界做“上下文垃圾回收”

长期单 Session 的问题不是“模型记得太少”，而常常是“模型被迫携带太多已经失效的信息”：旧测试日志、无效方案、已经修复的报错、重复工具输出和早期错误假设。它们不仅占 token，也改变注意力分配，造成 context rot。

Ralph 的做法是让 iteration 本身成为清理边界：结束当前 session，只把真正应该成为长期事实的内容写进 repository、Git、PRD、progress 或 tests；下一轮重新构造一个较干净的 context。snarktank/ralph 明确把每轮 fresh instance + clean context 作为核心机制。[2]

5.1 Context GC 的保留集与丢弃集

应保留 (Durable State)	通常应丢弃 (Ephemeral Context)
已提交代码、当前工作树	大段中间 reasoning
规格、验收标准、任务完成状态	已经解决的报错日志
发现的代码库约定与 gotcha	失败但已放弃的探索路径
可重复执行的 tests / scripts	重复文件读取结果
必要的决策记录	与当前 story 无关的历史上下文

更准确的理解

Fresh Context 不是“失忆”，而是把 memory 从无结构的 conversation history 迁移到有选择、有生命周期、可验证的 external state。

6. External Memory: 让环境记住，而不是让对话记住

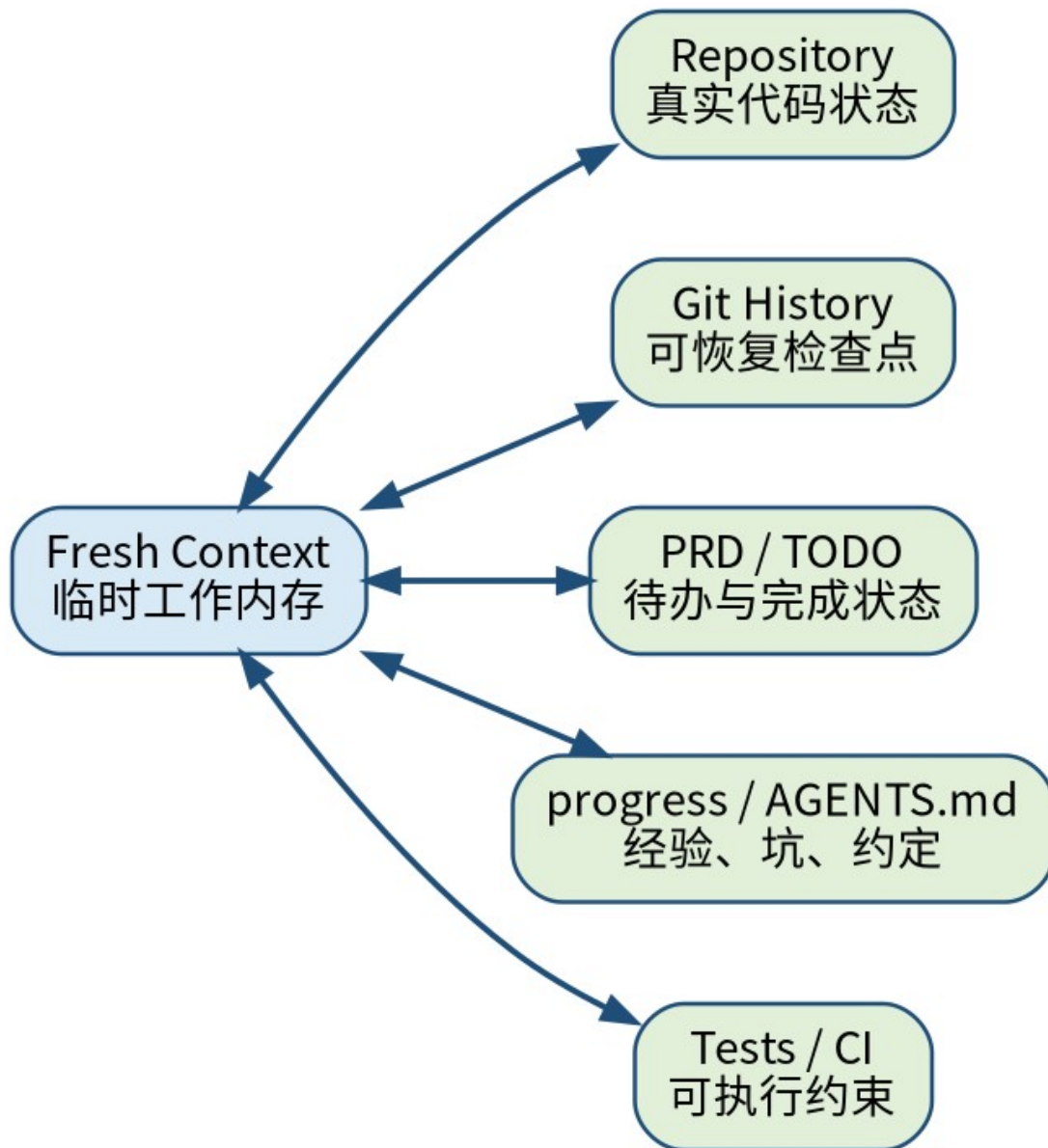


图 4 Ralph 的跨轮记忆来自外部状态，而不是无限 conversation history

External Memory 是 Ralph 能够 fresh restart 的前提。代码、任务状态、经验和验证约束分别存放在适合它们的载体中，新的 Agent 通过读取这些“锚点”恢复工作，而不需要重放之前整个聊天。

载体	保存什么	为什么重要
Repository	真实实现与配置	最终事实来源，不依赖模型叙述
Git History	检查点、diff、可回滚版本	支持定位“从哪一轮开始坏”
PRD / prd.json	任务列表、优先级、passes 状态	避免重复做已经完成的 story
progress.txt	跨轮经验、下一轮提示	低成本传递非代码上下文
AGENTS.md / conventions	稳定代码库规则与 gotcha	把发现变成未来默认约束
Tests / CI	可执行的行为要求	把“记忆”转成机器可验证约束

snarktank 的实现尤其强调 Git、progress.txt 和 prd.json 是 iteration 之间的记忆；其 README 还建议把发现的 pattern、gotcha 和 useful context 写回 AGENTS.md，使后续 Agent 自动获取。[2]

6.1 Memory 设计的一个重要原则：结论要尽量可执行

“不要忘记更新 Z” 写在 progress.txt 有用，但把它编码成 test、lint rule、schema constraint 或 acceptance script 更可靠。随着系统成熟，应尽量把自然语言记忆下沉成可执行约束，因为可执行约束更不容易被下一轮忽略。

7. Backpressure 与 Verifier：真正的质量控制面

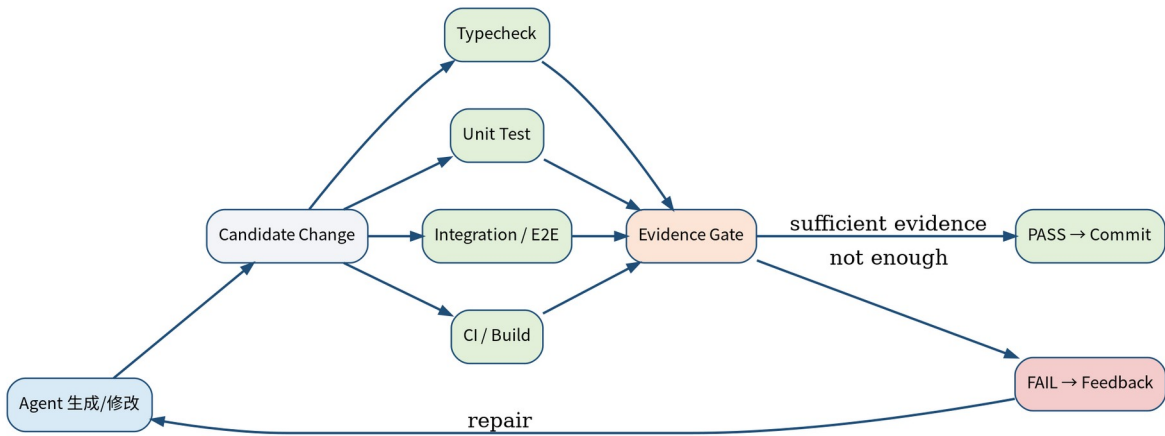


图 5 Backpressure：不是“多生成几轮”，而是每轮都必须面对可执行证据

Ralph 的关键不是 repetition，而是 feedback。没有 backpressure 的循环只会高速生成；有 backpressure 的循环才能把错误暴露、转成反馈并触发修复。Huntley 原始方法与后续实现都把测试、构建和受控验证看作核心约束；snarktank README 直接指出，typecheck、tests 与 CI 是 Ralph 能工作的必要 feedback loops。[2]

7.1 Verifier 的强弱决定 Ralph 的上限

验证层级	示例	可信度/风险
L0 自我声称	Agent 说“完成了”	极低；不能作为 DONE 证据
L1 静态检查	lint、typecheck、schema	可抓结构错误，但不代表行为正确
L2 单元测试	pytest / unit suite	局部行为较可信
L3 集成/E2E	真实服务、浏览器交互、接口链路	更接近用户行为
L4 Evidence Gate	当前 commit 绑定的多源证据 + 生命周期门禁	适合无人值守的高可信 transition

2026 年的 Proof-or-Stop 工作进一步提出：不要把 Agent 的“完成声明”直接当成 lifecycle state，而应要求新鲜、可追踪到源状态、机器可验证的 evidence 满足 gate 后，才允许进入 reviewed / tested / DONE 等状态。[4] 这与生产级 Ralph 的演进方向高度一致。

7.2 Verifier 也会错

更危险的是 verifier false positive：实现错误，但测试覆盖不足，结果被标记 PASS 并 commit；下一轮便会把这个错误提交当作新的事实基础。关于 verify-repair loops 的近期研究也指出，当 verifier 与 repairer 都有噪声时，继续 repair 并不保证单调变好，甚至可能破坏本来正确的候选，因此 stopping policy 需要考虑 verifier discrimination 与边际收益。[6]

设计原则

不要只问“Agent 能不能修”，要先问“系统能不能可靠地知道它修对了没有”。

8. 一轮 Ralph 的完整执行逻辑

一个工程化 iteration 可以拆成八个阶段。每一步都可以被观测、记录和设置边界。

1. Load: 读取 Spec、PRD、progress、Git 状态和必要代码库规则。
2. Select: 选择最高优先级且尚未通过的一个小任务。
3. Inspect: 在修改前搜索代码库，确认功能是否已经存在，避免重复实现。
4. Execute: 让 Coding Agent 在局部范围内读、改、运行工具。
5. Verify: 执行 typecheck、tests、build、E2E 或自定义 acceptance。
6. Decide: 依据证据判定 PASS、RETRY、REPLAN、NEEDS_HUMAN 或 FATAL。
7. Persist: PASS 后 commit，更新 story 状态，并记录可复用经验。
8. Restart: 结束当前 Agent session，下一轮以 fresh context 重建工作集。

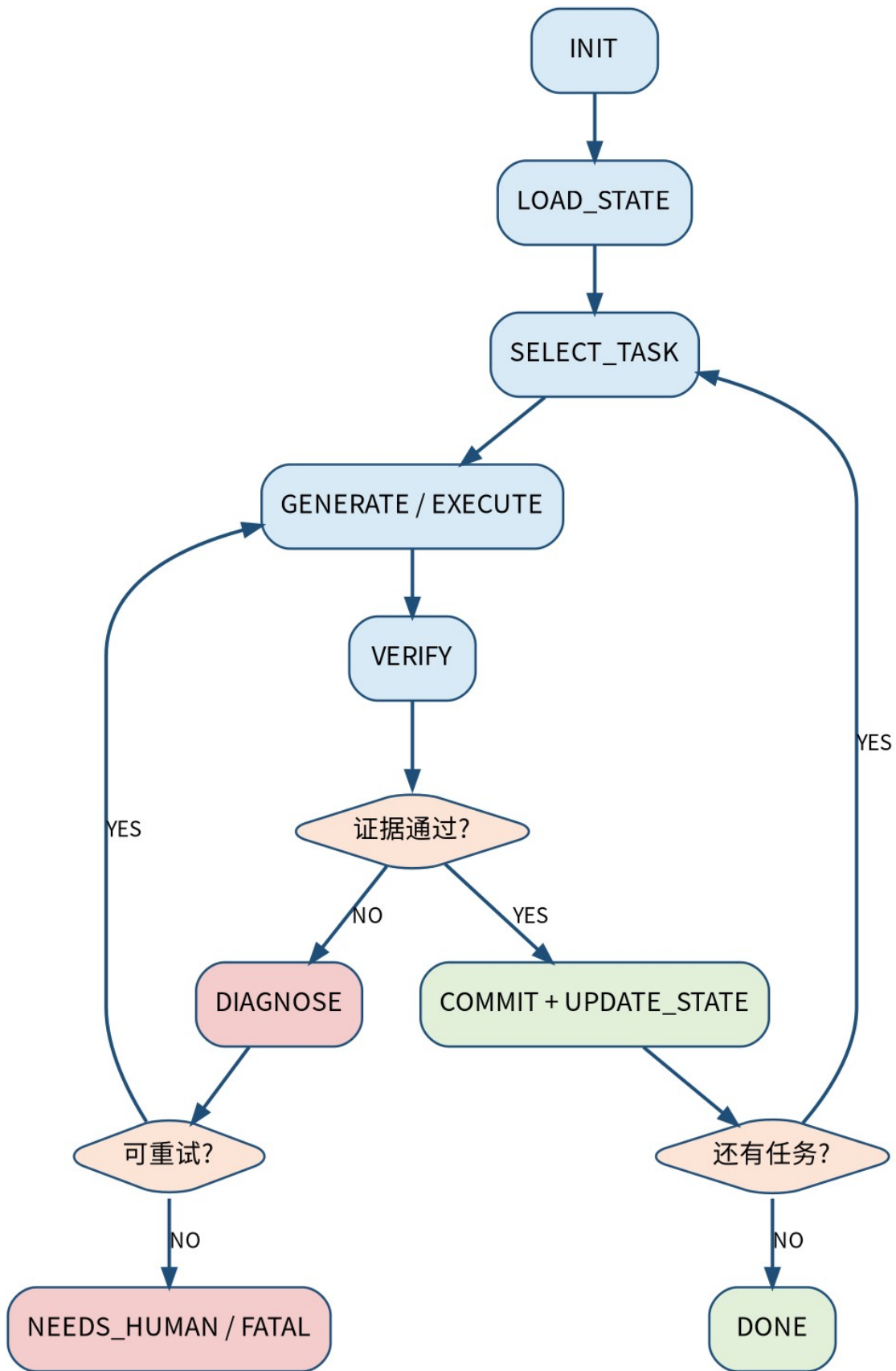


图6 一个生产级 Ralph 的建议状态机

8.1 为什么“先搜索再实现”是必要步骤

原始 Ralph prompt 中专门要求在修改前搜索 codebase，不要因为一次搜索没找到就假设功能不存在。[1] 这是为了抑制 Agent 的典型误差：把检索失败误判为事实不存在，然后重复实现相似组件，造成架构分叉。

8.2 为什么 commit 应该成为 iteration boundary

如果每个成功小任务都形成清晰 commit，Git 就天然成为 transaction log：可以 bisect、rollback、比较 diff，也可以把失败定位到特定 iteration。反之，十几轮修改堆在同一未提交工作树中，就失去了“哪一次状态转移引入问题”的可观测性。

9. Stop Condition、预算与生命周期状态

纯 while true 适合教学，不适合无人值守生产。企业级 loop 必须把“什么时候不再继续”设计成一等公民。

终止/暂停状态	触发条件	系统动作
DONE	所有 acceptance evidence 通过	标记完成、生成报告/PR
MAX_ITERATIONS	达到迭代上限	停止并保留当前状态
TIMEOUT	单轮或全局超时	终止进程，进入恢复/诊断
TOKEN / COST BUDGET	预算耗尽	停止或降级模型
NEEDS_HUMAN	需求歧义、权限、不可判定冲突	输出证据与最小人工问题
FATAL	环境损坏、不可恢复依赖/认证失败	终止并明确失败分类

snarktank 的参考实现以“所有 stories passes=true”作为成功条件，并设置 max iterations；满足条件后输出 COMPLETE 并退出。[2] 更成熟的生产系统应增加预算、超时、失败分类和人工升级状态。

9.1 停止策略不是细节

如果 verifier 有噪声，“固定跑 N 轮”可能让系统越修越坏；如果 stop rule 只依赖 Agent 自报完成，则会产生 false-DONE。近期 verify-repair 研究表明，停止策略应考虑验证置信度与继续修复的期望边际收益，而不是简单把“更多轮”视为更可靠。[6]

10. 为什么 Ralph 会有效：从控制系统角度理解

Ralph 可以视为一个带外部反馈的离散时间控制系统。Goal/Spec 提供参考信号，Agent 是具有不确定性的 controller，repository 是 plant/state，tests 与 verifier 是 sensor/evaluator，iteration boundary 负责状态提交与重新观测。

```
S[t] --Agent--> Candidate[t+1]
Candidate[t+1] --Verifier--> Evidence[t+1]
Evidence + Policy --Decision--> S[t+1] / Retry / Stop
```

10.1 Persistence > One-shot Intelligence

大任务一次性成功的概率可能很低，但“小任务 + 强验证”的单轮成功率可以高得多。Ralph 把目标从“模型一次写对整个系统”改成“每轮取得一个可证明、可提交的局部进展”，最后通过状态累积逼近整体完成。

10.2 环境提供真实反馈，抑制语言模型自洽幻觉

LLM 很擅长生成一个听起来完整的解释，但代码是否真的编译、接口是否真的工作、页面是否真的可点击，不应由同一个语言生成过程自我裁决。Ralph 把这些判断下沉到编译器、测试框架、浏览器或其他环境反馈中，从而将“语言上的合理”转化为“环境中的证据”。

10.3 迭代把失败变成可学习的 failure domain

当某类失败反复出现，工程师不应只在当次 prompt 中提醒，而应修改 loop：增加 test、更新 AGENTS.md、调整 search rule、限制并行构建、增加 timeout 或改变任务粒度。这样，人的工作从逐步执行任务转向“编程反馈系统本身”。Huntley 后续文章把观察失败域并让同类问题不再发生视为循环工程的重要学习方式。[5]

11. Ralph 的主要失败模式

11.1 Verifier 假阳性：最危险的失败

错误实现如果被弱测试误判为 PASS，会被 commit、写入进度并成为下一轮的输入事实。由于 Ralph 强调持续状态演进，这类错误不是孤立错误，而会形成 compounding error。



图 7 错误复利：错误一旦通过 gate，就会成为后续 iteration 的地基

11.2 任务过大：单轮无法闭环

当 story 大于一个 context window 可可靠完成的范围时，Agent 往往同时留下未完成实现、未清理日志和部分测试；下一轮虽然 fresh，但读取到的是一个模糊的中间状态。解决办法不是增加无限上下文，而是重新切分 work unit。

11.3 Context 锚点污染

progress.txt 或 AGENTS.md 也可能积累过时、矛盾或过度具体的经验。如果所有历史“教训”永久保留，external memory 会重新变成另一种 context rot。成熟系统需要 memory lifecycle：写入门槛、合并、去重、失效和版本化。

11.4 重复实现与错误搜索

Agent 的代码检索不是完美数据库查询；一次 grep 失败不代表能力不存在。应对方式包括多路径搜索、符号/LSP 查询、依赖扫描和“修改前先证明不存在”的检查步骤。

11.5 Reward Hacking / Test Gaming

如果目标函数只有“tests green”，Agent 可能选择修改测试、绕过检查、硬编码结果或缩小断言，而不是实现真实需求。应把 acceptance evidence 与业务约束、测试来源权限、禁止修改区域等一起设计。

11.6 无界循环与成本失控

无限 iteration、无 timeout、无 token budget、无 error taxonomy，会使网络/权限/模型故障被误当成“代码需要再修一轮”。生产实现必须区分环境失败和实现失败。

12. 从 Pure Ralph 到 Production Loop



图 8 Ralph 的工程成熟度：从一次 Prompt 到 Graph of Loops

阶段	新增能力	适用范围
L0 一次 Prompt	一次 Agent session	小型、明确任务
L1 Pure Ralph	外层重复启动	教学、快速实验

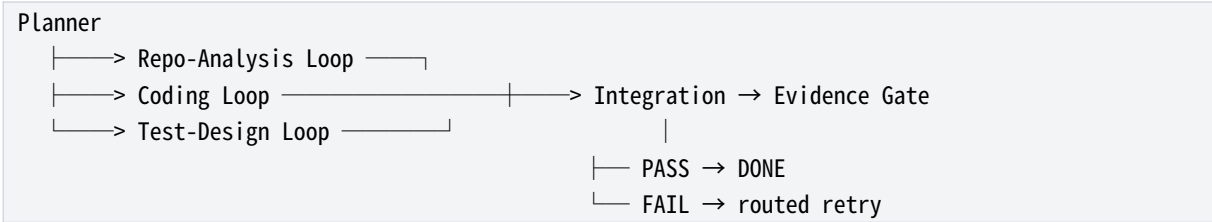
L2 Stateful Ralph	PRD + Git + progress + fresh context	中等长任务
L3 Verified Ralph	强 tests / E2E / acceptance evidence	可无人值守的局部开发
L4 Production Loop	状态机、恢复、预算、失败分类、观测	公司内部长期运行
L5 Graph of Loops	多 loop 路由、fan-out/fan-in、局部上下文	大型复杂项目与软件工厂

12.1 Production Loop 最少应该补什么？

- 持久化 state.json：iteration_id、task_id、status、retry_count、last_good_commit。
- 进程级 timeout + provider timeout + 网络/认证/模型错误分类。
- 每轮前后 Git checkpoint，并确保 crash 后可以恢复到确定状态。
- 结构化 eval-result.json，而不是依赖 Agent 自然语言总结。
- acceptance script / test matrix 作为 Evidence Gate。
- 全局 MAX_ITERATIONS、token/cost budget、NEEDS_HUMAN terminal state。
- 完整 trace：输入锚点、模型、token、工具、命令退出码、验证证据、状态转移。

13. Ralph 与 Graph Engineering: Graph of Loops

Ralph 在图论上可以看成是一个很小的 cyclic state graph：Generate → Verify → Decision → Generate。
Graph Engineering 不会否定 Ralph，而是把多个局部循环、确定性节点、验证节点与路由关系组合在更大的执行拓扑中。



最合理的演进不是把 Ralph 推翻重写，而是把稳定的 Ralph-style loop 作为 Graph Node：节点内部继续用 fresh context + state + verification；Graph 外层负责并行、依赖、fan-in、failure routing 和 human gate。

13.1 什么时候保持单 Loop，什么时候升级 Graph？

场景	推荐	原因
修一个 failing test / 局部 feature	单 Ralph Loop	依赖简单，Graph 开销不值得
单仓库中连续完成几十个小 story	Stateful/Verified Ralph	顺序推进最自然
多个相对独立模块可并行	Graph of Loops	可利用 fan-out/fan-in
失败需要回到不同责任域	Graph of Loops	测试失败回 Coder，架构失败回 Planner
多 repo / 多环境 / 多角色审批	Graph Runtime	需要显式 topology 与状态路由

14. 实战设计清单与推荐架构

14.1 开始一个 Ralph 任务前

- 把需求写成可检查的 Spec，而不是一句模糊目标。
- 将任务拆成单轮可闭环的小 story；每个 story 明确 acceptance criteria。
- 定义哪些文件/测试/命令是 source of truth。
- 明确 Agent 允许修改和禁止修改的范围。
- 准备 fresh context 所需的最小锚点：spec、当前任务、repo rules、progress、tests。

14.2 每个 iteration 必须记录

字段	建议内容
iteration_id	单调递增，支持日志关联
task_id	当前唯一 story / issue
base_commit / result_commit	输入和输出 Git 状态
model / config	模型、thinking、tool/harness 配置
evidence	测试命令、退出码、报告文件、E2E 证据
decision	PASS / RETRY / REPLAN / HUMAN / FATAL
failure_class	CODE / TEST / ENV / AUTH / NETWORK / PROVIDER / TIMEOUT
cost	token、时长、重试次数

14.3 推荐的生产级目录

```
project/
├── specs/
├── src/
├── tests/
├── AGENTS.md
├── .loop/
│   ├── state.json
│   ├── prd.json
│   ├── progress.md
│   ├── gotchas.md
│   ├── eval-findings.md
│   ├── eval-result.json
│   └── acceptance-script.sh
└── .git/
```

架构建议

把 Git 当 transaction log，把 .loop/state.json 当 control-plane state，把 tests/acceptance 当 evidence plane，把 Agent conversation 当可丢弃的 execution cache。

14.4 一套推荐的决策顺序

9. 先判断任务是否足够小；太大先拆，不要直接增加 iteration 数。
10. 再判断 verifier 是否能覆盖真正需求；不能验证的任务不要直接全自动。
11. 再设计 persistent state 和 rollback point。
12. 最后才选择模型、prompt 和并发数。模型是执行器，不应替代控制面设计。

15. 结论

Ralph 的历史意义不在于发明了 while loop，而在于把 Coding Agent 的使用方式从“人类一步步 prompt”推向“工程师设计外部反馈系统”。它把 session 视为可抛弃的执行单元，把 repository 与测试环境视为长期现实，把 iteration 视为状态转移边界。

真正可靠的 Ralph 不是 Prompt × 100，而是 Goal + Small Task + Fresh Context + Persistent State + Backpressure + Stop Condition。它允许 Agent 犯错，但要求系统能够发现错误、限制错误传播，并以可恢复状态进入下一轮。

因此，评价一个 Ralph 系统最重要的指标不应只是“能连续跑多少轮”，而应是：每轮是否有可验证的净进展、错误是否能够被 gate 拦截、失败是否可定位、状态是否可恢复、停止是否有依据。达到这一层后，Ralph 才真正从脚本升级为 Agent Runtime。

最终判断

Ralph = 一个以 fresh context 驱动、以外部状态为记忆、以验证证据为反馈、以明确终止态为边界的长运行 Agent 控制循环。Graph Engineering 则是在此基础上，把多个这样的局部闭环组织成更大的可控执行网络。

参考资料

- [1] Geoffrey Huntley. “Ralph Wiggum as a software engineer.” 2025. ghuntley.com/ralph/（原始 Ralph 方法：monolithic、one item per loop、spec/context、search before change 等）
- [2] snarktank/ralph. GitHub repository / README. 2026-08-13 访问。（fresh instance、Git + progress.txt + prd.json、workflow、feedback loops、stop condition）
- [3] Sandeco Macedo. “Stop Hand-Holding Your Coding Agent: Engineering the Loops that Replace Step-by-Step Prompting.” arXiv:2607.00038, 2026.（loop specification: trigger、goal、verification、stopping rule、memory）
- [4] Jek Huang et al. “Proof-or-Stop: Don’t Trust the Agent, Trust the Evidence — Loop Engineering for Verifiable Evidence-Gated Lifecycle Control.” arXiv:2607.14890, 2026.
- [5] Geoffrey Huntley. “everything is a ralph loop.” 2026-01-17. ghuntley.com/loop/（从任务自动化扩展到编程 loop 本身、观察 failure domain）
- [6] Yitao Wu et al. “Verify, Repair, Repeat, or Stop? Robust Stopping for Noisy Verify-Repair Loops in LLM Agents.” arXiv:2607.17641, 2026.（noisy verifier/repairer 与 stopping policy）

注：本文对 Ralph 的工程扩展（如 failure taxonomy、预算状态、evidence plane、Graph of Loops）包含基于上述原始材料与近期 Loop Engineering 研究所做的系统化推演，属于工程建议，不应误认为所有字段均来自某一个 Ralph 官方规范。

附录 A：Ralph Loop 的极简伪代码

```
state = load_state()

for iteration in range(MAX_ITERATIONS):
```

```

task = select_one_task(state)
if task is None:
    return DONE

ctx = build_fresh_context(
    spec=state.spec,
    task=task,
    repo=current_repo(),
    progress=state.progress,
    rules=load_agent_rules(),
)

candidate = run_agent(ctx)
evidence = run_acceptance(candidate)

if evidence.pass_gate:
    commit = git_commit(candidate)
    state = mark_task_done(state, task, commit, evidence)
elif evidence.retryable:
    state = record_failure(state, task, evidence)
else:
    return NEEDS_HUMAN

persist_state(state)

return MAX_ITERATIONS

```

这段伪代码里最值得关注的并不是 `run_agent`，而是其前后的 `build_fresh_context`、`run_acceptance`、`git_commit`、`persist_state` 和 `terminal states`。这些部分才是 Ralph 从“调用模型”走向“工程系统”的核心。

附录 B：审查一个 Ralph 实现时的 12 个问题

13. 每轮是否只追一个明确、可闭环的主要任务？
14. 新一轮是否真正 fresh，还是仍携带大量历史 session 噪声？
15. 跨轮哪些信息被持久化？它们是否有清晰 source of truth？
16. 成功由谁判定：Agent 自报，还是外部 evidence？
17. 验证是否覆盖真实业务行为，而不仅是“命令退出 0”？
18. 测试/验收本身是否允许 Agent 随意降低标准？
19. 每轮是否有 Git/状态检查点，可以单独 rollback？
20. 网络、认证、provider timeout 会不会被错误地算作代码失败？
21. 是否存在 MAX_ITERATIONS、timeout、token/cost budget？
22. 遇到不可判定问题时是否有 NEEDS_HUMAN，而非无限重试？
23. progress / memory 是否会老化、矛盾和无限膨胀？
24. 是否有足够 observability 解释“为什么这轮从 A 转到 B”？